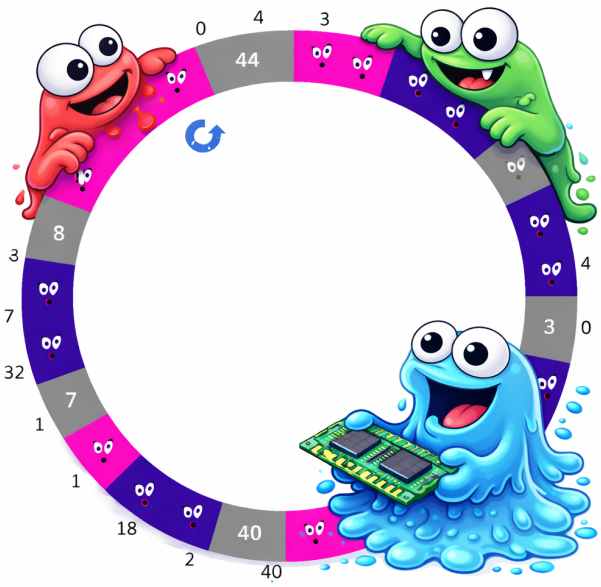
RINGwars Documentation

Grundpraktikum Programmierung

B. Sc. Informatik

Fakultät für Mathematik und Informatik
FernUniversität in Hagen

Name: Shogo Seifert
Matrikelnummer: 7172397
Lehrgebiet: Kommunikationsnetze
Professor: Prof. Dr. Herwig Unger
Abgabedatum: 04.01.2025



FernUniversität in Hagen

Contents

1 Introduction **3**

2 Strategy **4**

2.1 Initial Thoughts and Strategy Planning 4

2.2 Move Categories 5

2.2.1 Abstract Moves 5

2.2.2 StepOne Moves 5

2.2.3 Attack Moves 5

2.2.4 Defensive Moves 6

2.2.5 Expansion Moves 6

2.2.6 Colony Moves 7

2.2.7 Strategy for Visibility <= 2 7

2.3 Final Strategy 7

2.4 Agent Model and Task Environment 8

2.5 Experimentation with dynamic and subtle decision making 9

2.6 Approaches to long-term documentation and evaluation of the agent 10

2.7 Simulation approach 10

3 Architecture **11**

3.1 Layered Architecture 11

3.2 Overview 12

3.3 Object Oriented Design Principles 13

3.4 Budget System 13

3.4.1 Motivation 13

3.4.2 Implementation 14

3.4.3 Challenges 14

3.5 Important Methods 14

3.6 Core Data structures 15

3.6.1 Node 15

3.6.2 GameModel 15

3.6.3 ArrayList 15

3.6.4 HashMap/Budget 15

4 Analysis and Performance **16**

4.1 Behavior in different Game Situations and against other agents 16

4.2 Strengths and Weaknesses 17

5 Conclusion and Future Work **17**

6 Sources **18**

A Appendix **19**

A.1 Flowchart Decision Making Agent.FINAL 19

A.2 Class Overview 20

A.3 takeFerniesFromBudget() 21

A.4 getBestBorderPlacement(Node enemyNode, GameModel gm) 22

A.5 fightStrongestEnemiesBestPlacement() 23

A.6 Winning Rates 24

1 Introduction

This document describes the RINGwars agent developed as part of the Programmierpraktikum in the winter semester of 2025/2026. RINGwars is a turn-based two-player strategy game, played on a circular game board consisting of discrete fields, called nodes. The players are competing directly with each other, the gains for one player imply losses for the other. The objective of the game is to conquer and eliminate all enemy-controlled nodes on the game board.

From an implementation perspective, the project presents different challenges. These include designing a proper software architecture, parsing and processing the step files by the game engine, storing and analyzing the game state and generating valid move sequences without producing illegal moves. Furthermore, the agent has to be able to manage limited resources, appropriate data structures have to be chosen and the move files have to be written strictly to the required format.

The primary goal of this project was to solve these problems in an appropriate way while developing a stable and competitive agent.

An "agent" in the sense of AI, as described in the book *Artificial Intelligence - A Modern Approach* by Stuart Russell and Peter Norvig, is a program where we expect it to "operate autonomously, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals." [1, p.4]. The question arose as to which type of agent, with which abilities and characteristics, would best fit the RINGwars game. The paper works out this question and describes the agent and its development

The ***Strategy*** section outlines the development of the agents strategy over time. It outlines the initial considerations, the categorization of different types of moves and the final strategy, how the decision making works and why other ways of implementing the agent, such as a simulating or learning agent, were not pursued.

The section ***Architecture*** focuses on the technical realization of the strategy. It starts with an explanation of the layered architecture, an overview of a class diagram and a short description of the classes, discusses design decisions and how concepts of object-oriented programming such as modularity, encapsulation, and separation of responsibilities were applied to fulfill the agents requirements.

The ***Analysis and Performance*** section examines the behavior of the agent in different game settings and against different opponents. It also reviews the strengths and weaknesses of the strategy.

Finally, ***Conclusion and Future Work*** reflects on the project and outlines possibilities of improvements and extensions of the agent.

2 Strategy

2.1 Initial Thoughts and Strategy Planning

At the beginning of the project, the first step was to gain some more insights and details about the game mechanics. The game has both partly-deterministic elements (to a certain extent local fights, border fights), as well as non-deterministic elements (resolution of triple-fights, consequences of the visibility). As long as the agents cannot see the whole board, we can classify the game as an "Imperfect Information Game", which means that the players have to guess and make uncertain decisions. As a consequence, the most locally optimal move does not mean that it is necessarily the globally optimal outcome.

After determining the characteristics of the game, we had to consider which type of agent would be most suitable to implement. What is certain is that the agent would have to be rational. In the book *Artificial Intelligence - A Modern Approach* a rational agent is described as an agent, *"that acts so as to achieve best outcome or, when there is uncertainty, the best expected outcome"*[1, p.4].

Due to the possibility of rapid movement and Fernie placement, the game is highly dynamic and situational. As we compare it to other classical turn-based two-player strategy games like chess, some fundamental differences become apparent. In the RINGwars Game, it is possible to do several moves in one turn and moves of both players are done simultaneously. Also, the visibility can be limited, which means that the real power of the enemy can only be guessed. Furthermore, the strength of a node is not fixed and it depends on the number of Fernies placed on it. Due to this characteristics, the conclusion was that RINGwars is not perfectly suited for deep search techniques. The combination of imperfect information, non-deterministic battle resolutions, simultaneous moves and an extremely large action space and many different ways of playing the game would result in an extremely large and uncertain search tree. As a result, this project focuses on building an agent with **limited rationality**, which means the agent will make decisions that are rational with the constraints of the given information and time.[1] A heuristic-based, reactive approach was chosen, focusing on locally optimal decisions rather than global optimization or long-term planning strategies. The goal was to implement a flexible and responsive agent that demonstrates a high win rate and stability.

The development of the agents strategy did not follow a predefined theoretical master plan, like a strategy you would have found in nature's survival instincts or an implementation of a specific well-known algorithm. Rather, after reading the slides carefully, the foundation was an approach based on how a human would play the game. Then, the agent developed through iterative and experimental approach, which led to an effective strategy but also required significant development effort.

From a software architecture perspective, my goal was to develop an agent and overall system that fulfills principles of object-oriented programming: modularity, extensibility, clear responsibilities, encapsulation and abstraction[2]. Thanks to the modularity of the agent and the parametrization of the move-methods, it was possible to implement and test several different strategy orientations during the development, such as colony-based, defensive, aggressive and expansion-based.

Although those strategy approaches were not developed in isolation to fully extend, the observation and exploration led to the conclusion that it was suboptimal to rely on a single strategy but rather to combine certain approaches of these strategies and to implement a flexible agent.

Therefore, the moves are categorized into different "styles" and there are several possible moves for each category. The following sections briefly describe those categories and then explains the final strategy as a combination of moves from various of those categories. Interestingly, the most effective agent did not require a lot of different moves or highly-complex decision making. Instead, it turned out that a more simple but robust strategy was the most beneficial.

2.2 Move Categories

2.2.1 Abstract Moves

This component serves as the parent class of all move categories. It contains a collection of basic, reusable, and fallback actions, which all more specialized move classes can access and reuse. All other move categories inherit from this abstract class in order to avoid code duplication and to ensure robustness of higher-level moves. It provides basic fight moves, fallback moves that guarantee that no previously taken Fernie is unused, and moves for placing and distributing new Fernies. The class contains moves like the following. For the clarity and readability, method parameters are left out in this paper most of the time.

putNewFerniesSmart()
Distributes new Fernies among owned nodes in an efficient and capacity-aware manner, while firstly putting it on the node with the highest number of Fernies until it is full.

putAbsFerniesSmart(Map<Integer, Integer> budget, int absFernies)
Places a given number of Fernies (often the remaining Fernies of previously taken ones) efficiently on owned nodes.
fightEnemy()
Executes an attack against a specified enemy node, including the necessary budget handling and placement logic.

2.2.2 StepOne Moves

This class contains opening moves that are used exclusively in the first step. Like all following move classes, it is a subclass of "AbstractMoves". In RINGwars, the first move and early game is particularly important, since it influences strongly the visibility and ownership bonus and early territorial control. Depending on the ring size, the enemy's behavior, and the visibility, the effectiveness of a specific opening move can differ. If the visibility allows it, some opening moves can already win battles on the edge of the visibility range in the first move.

expandStepOneOriginal()
This is the original method for step one. It expands on both directions up to the maximum of the visibility. It gradually thins out the Fernies from the innermost to the outermost node at the very outer border.

expandStepOneMaxEdge()
This method places all Fernies on one side of the edge of the visibility boundary and places one Fernie on all other initially visible nodes (except one node) to get the most of the ownership bonus.

2.2.3 Attack Moves

This class collects all actions that attack enemy nodes directly. The main objective of these moves is to reduce the amount of Fernies and nodes of the opponent. The main challenge when attacking an enemy node is that its power should be calculated not only by the number of Fernies on the node itself, but also its neighbors on both sides. This is because fights are resolved in two steps, and an initially won local fight could turn out to be ineffective, when a border fight afterwards results in a loss of all Fernies that were attacking. Furthermore, the outcome can differ significantly whether the fight is executed as a local or as a border fight. While in local battles the difference between the Fernies is calculated, in border fights, the winner absorbs the enemy's Fernies. For this reason, border fights and triple fights are generally preferred. Based on those considerations, the attack moves are divided into two main categories:

Neighbor-based attacks, which calculate the effective enemy strength as the sum of three nodes: the target node itself plus the enemy's Fernies of its directly adjacent neighbors.

Best-placement attacks, which determine the best placement for attacking an enemy node. This means the moves evaluates whether the attack should be done as a border fight or as a local fight and from which side it is the most advantageous in winning Fernies.

Both approaches can be applied either against a single enemy or a list of enemies, sorted by strength in ascending or descending order. Implementations of these methods are for example:

fightStrongestEnemiesWithNeighbors()
fightStrongestEnemiesBestPlacement()

and their corresponding variations for attacking weaker enemy nodes first. The moves could also have been implemented as a single move method and parameterized, but separating them into distinct moves allows the programmer to directly understand its tactical intention and increases readability and clarity.

2.2.4 Defensive Moves

This category collects all methods that have a defensive intention. These include, for example, the following methods:

runAway()
Finds the best node to retreat from the opponent. It prefers hidden nodes, which makes it harder for the enemy to calculate the actual strength of the agent.

collect()
Collects Fernies from all other nodes to shifts them on the strongest node. This move is particularly effective in late-game situations when both players control only a few nodes and all nodes owned by the agent are relatively weak.

2.2.5 Expansion Moves

This category collects methods that aim to expand the agents territory by occupying neutral nodes and enemy nodes that are weak. The expansion increases the ownership bonus and visibility, providing more insights into the enemy's actual strength and nodes placing while making it harder for the enemy to expand itself. The most important expansion moves are:

getAsManyNeutralNodes(budget: map, perNode: int)
Distributes Fernies across as many neutral nodes as possible. As a parameterized method, the agent can decide dynamically how many Fernies per neutral node should be put.

getAsManyNodesAndFight()
Combines expansion and offense by attacking weak enemy nodes and occupying neutral nodes.

2.2.6 Colony Moves

This category collects methods that relate to colonies, which are defined as continuous connected nodes that are all owned by the same owner. They aim to strengthen already established connected nodes instead of focusing on individual nodes. Typical colony-based actions are the consolidation of Fernies within strong colonies or the defending of the boarders. The class includes:

stableColony()
This method redistributes the owned Fernies onto the strongest (in terms of Fernie count) colony, making it more resilient against attacks of the enemy.

defendColony()
This method attacks all opponents that are located within a distance of two nodes from a colonies border.

2.2.7 Strategy for Visibility $<= 2$

For this edge case, the agent places one Fernie on each visible neutral node and all other Fernies on the edge of the resulting colony on each step. The idea is that the agent would benefit from the ownership bonus while winning battles against enemy nodes with one strong node. If the strongest node is full, the new Fernies get placed on the neutral nodes.

It should be noted that the categorization of the moves can not be always rigid, since the can belong to multiple categories. The moves of the class ColonyMoves were explored during the development but they are not part of the final agent. Nevertheless, they remain as a foundation for future work and architectural artifacts.

2.3 Final Strategy

During the process of developing my agent, I experimented with different combinations of moves from all previously described categories and observed the strengths and weaknesses. After extensive testing of different move combinations, I came to the conclusion that a highly aggressive yet flexible mixed-strategy is the most effective approach. In order to react flexible to different game settings, the agent adapts depending on some key parameters. The two most influential parameters are the ring size and the visibility.

The agent categorizes the ring size as follows:

SMALL	**MID**	**LARGE**
≤ 20	≤ 30	> 30

In each category, the agent applies slightly different moves or parametrization. In the category "SMALL", the agent prioritizes attacking the strongest enemy nodes first and is very aggressive, by attacking with 110% of the enemy's strength. On small rings, expansion is less effective, so attacking the stronger nodes first to weaken the enemy is beneficial.

In the categories "MID" and "LARGE", the agent focuses on attacking weaker enemy nodes, attacking strong enemy nodes after that as border fights to absorb Fernies and then expanding by occupying neutral nodes.

Across all categories, the agent relocates the remaining Fernies of its previously strongest nodes to another node, preferably one that is hidden from the enemy. The intention behind it is that

aggressive opponents typically attack the strongest visible node. When the agent takes Fernies of its strongest node to attack the enemy, it gets weakened and thus an easy target for the enemy. For this reason, it transfers the remaining Fernies to another node, preferably a hidden node, which makes it more difficult for the enemy to calculate the agents actual strength. Independent of the ring size, the agent follows these core strategical principles:

- to aggressively attack the enemy as much as possible, especially the weaker nodes, to improve visibility, increase the ownership bonus, and reduce it for the enemy and its overall strength
- to expand on neutral nodes by placing a minimal number, (e.g., one or two), to get ownership bonus and to gather information about the enemy
- efficiently distributing new Fernies once the strongest node has reached the maximum capacity per node
- relocating the strongest node so that a potential enemy attack on it will not be effective
- to strengthen the strongest node that the enemy cannot attack or defeat it that easily

The ideal move would be to combine all of these objectives. Naturally, these objectives, contradict which each other to a certain amount and they have to be specified according to the ring size, the visibility and the opponent. Although the ownership bonus is a crucial influence for the strategy, it is hard to calculate it precisely, because it has to be distinguished from the normal new Fernies per turn. For this reason, I did not implemented it within my agents strategy as a decision parameter.

Finally, the agent adapts its strategy against opponents that concentrate most of their Fernies on only a few nodes. If the number of visible enemy nodes is very small (three or fewer), the agent responds with more aggressive attacks, assuming that the enemy is investing the new Fernies into these few nodes. In the appendix, a flowchart of the decision making of the final agent is provided.

2.4 Agent Model and Task Environment

After the considerations presented earlier and defining its strategy, in this section, the agent model and its environment is classified in AI theory terms and set apart to different agent models. As illustrated in the book *Artificial Intelligence A Modern Approach*, an agent can be defined with the PEAS framework. **PEAS** stands for **P**erformance, **E**nvironment, **A**ctuators and **S**ensors. In the case of RINGwars, we would consider different possibilities for the performance measure, but the most simple and important is that the agent has to win the game. The following table describes each part examined specifically for the RINGwars agent.

Performance Measure	**Environment**	**Actuators**	**Sensors**
• Winning the game • Eliminating the opponent • Avoiding illegal moves • Maximizing ownership bonus	• The ring • Game rules • Other agents • partial observable • dynamic • competitive	• Placing Fernies • Moving Fernies • Writing move commands • → **MoveWriter**	• Step files (reading, analyzing and storing the game state) • Perceived game history • → **StepReader**

Table 1: PEAS description of the task environment for a RINGwars agent

With the PEAS measurement in mind, we can further think about the agents model main design, which is the perception and the action mapping. As outlined in the earlier sections, through

analyzing the game environment and iterative experimentation, the conclusion to implement a simple reactive and heuristic based agent, which acts as rational as possible, seemed to be the best solution and offered a desirable trade-off between performance, complexity and robustness. An agent in each form is mapping the input it gets to an action it executes. It has to react rationally based on the information it has available. As a **simple-reflex agent**, with heuristic state abstraction, the agent follows a condition-action rule, a perception-action mapping, based only on the explicit game state it gets for the current and previous step, without long-term planning or simulation. Also because of the partial observability, it is not omniscient, which means, the agent does not have the full information of the game. The input of the mapping is the current game state and the previous step, with the ring as the foundation of the information. The ring gets then analyzed and actions executed. Although the resulting operations follow similar patterns, an aggressive agent proved to be the most effective strategy in practice. The StepAnalyzer also compares the last step with the current step to rate the enemy's behavior and evolutions. Considering that, the agent could be defined as partly a **model-based** agent.

But since since it has no internal model of the world beyond the current and the last game state, the agent is not fully **model-based**. It is also not **goal-based** since it does not have an explicit predefined goal and takes precedence over the actual action in each step. Moreover, it is not a **utility-based** agent, as it does not calculate or evaluate multiple actions and chooses a move based on the expecting outcome. Finally, it is not a **learning agent** as it does not adapt its actions based on experience gathered over a history of steps or statistics accumulated over games.

In the following subsections, some alternative approaches and agent models are shortly discussed and explained, why they were not pursued further for this project for now.

2.5 Experimentation with dynamic and subtle decision making

Within the process, I have tried many different decision making approaches. The StepAnalyzer makes it possible to compare the current step with the previous step in terms of changes in the strength and comparison to the enemy. It also features four methods for rating the enemy's behavior in terms of defense, aggression and expanding.

Experimentation with more dynamic and subtle decision making on information like that did not improve performance compared to simpler decision making. In fact, a more simple decision making was even slightly more effective. An outcome of comparing results of playing with different decision makings against different agents was that a decision making based on information and enemy behavior actually reduced the number of winning matches.

In the appendix, the results of Agent.KOMPLEX, which has a more nuanced and dynamic decision making by rating the enemy's behavior, is shown in a table. In comparison to the Agent.FINAL, also shown in the appendix, it has lower rates of winning games. Agent.FINAL wins overall 73% of the games, whereby Agent.KOMPLEX wins 70%. Especially against Agent Y, the complex agent has a significant lower winning rate of 14%, while having a better winning rate of 7% against the Agent Z, resulting in better performance on small rings. Of course, one of the reasons might be the way of deciding, but in the end, I came to the conclusion that a few proven and aggressive moves are sufficient to develop a good performing agent and that more complexity does not mean better performance. Subtle decision making is very hard to figure out, since the move methods themselves already have a few parameters to change. The decision space becomes noisier and heuristic ratings introduced additional uncertainty. As a result, the use of an observation of the development of a game and decision making based on historical game changes (e.g. information about the enemy's behavior) was considered limited. Additionally, the agent can get information about the play style of the enemy indirectly also in easier observations. These conclusions are based on qualitative observation and experimentation rather than large quantitative evaluation of statistics.

2.6 Approaches to long-term documentation and evaluation of the agent

One considered approach was to record all games and moves against different opponents while rating each step, move or strategy. After evaluating, these could potentially be used to hard coded the agents strategy based on that information. Thus, the documentation would only serve as a preparation and tool for the implementation of the agent and not be analyzed/read each game, which then would be according to the rules to only read the move and step files. Implementing and maintaining an evaluation framework would be highly increase the systems complexity. Additionally, the past experiences with ratings did not impvrove the performance. For these reasons, I decided against this approach.

2.7 Simulation approach

Implementing famous algorithms based on simulations like MINIMAX, MCTS or the Alpha-Beta pruning were considered, too. However, RINGwars is a very dynamic and situational game with a huge action space and extremely large possibilities for simultaneously moves. Furthermore, the agent has part wise only imperfect information and the game has non-deterministic battle resolutions. In case of RINGwars, this would lead to an incredible large number of possibilities and an extremely fast growing search tree combined with uncertainties, which would be difficult to handle and would probably slow down the reaction time of the agent in observable manner. Besides, experimenting with the agents current heuristic and reactive-based strategy led to satisfactory outcome. Again, implementing one of those algorithms would increase the complexity of the program to a significant amount without knowing if the outcome would make a substantially difference.

3 Architecture

3.1 Layered Architecture

The implementation works with a layered architecture, where each of the 4 layers has a specific responsibility and task.

The Input/Output Layer controls the basic processing of the program and is responsible for the reading and writing of the .txt files. The layer is completely separated from logical or strategic decision. Because of the interfaces, it is suitable for changes, for example, if the structure of the game board changes. The Input/Output layer contains the IReader, IMoveWriter and the RINGwars_7172397_Shogo_Seifert class.

The Data Layer contains pure structural state data and facts about the game board of one step (and eventually the step before). Those data where previously read in in the Input/Output layer. This layer also stays independent from strategy decisions. This Layer includes the GameModel and the Node class.

The Analyze / Utility Layer evaluates and analyzes data and provides knowledge, but also in this layer, there is no decision making for the strategy or the moves. For example, methods of GameUtils class supplies information about the strongest enemy node, it updates the budget (see 3.4.), analyzes distances and neighbors, etc. Many methods are reused by other move methods, such as for generating a list of opposing nodes. This reusability prevents redundancy and simplifies the move methods. This Layer consists of the StepAnalyzer and some GameUtils helper methods.

The Strategy Layer, is the actual "brain" of the agent, where decisions are made. The StrategyDecider is the Top-Level decider. Because of the possibilities of combining different methods in the StrategyDecider, it is very flexible. This layer contains the StrategyDecider and the Move-classes. Each move returns a move as a string in acceptable form, which is then gets combined in the StrategyDecider which forwards it to the IMoveWriter.

Each layer communicates with its neighbor layers and they are ordered linearly. This architecture was very helpful and useful to adapt to the changes of game settings, different agents and to experiment with strategies using StrategyDeciders.

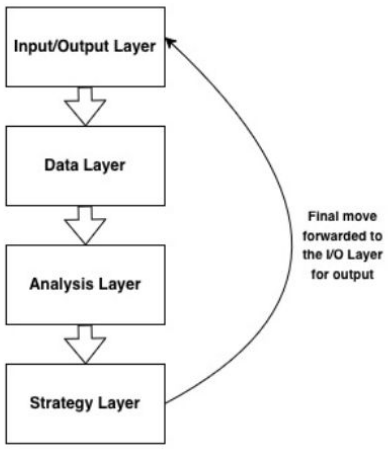


Figure 1: Architectural Layers**Architectural Layers**

3.2 Overview

The agent consists of different classes. The following provides an overview of the classes with a brief explanation for each class. Abstract classes/parent classes are written italic, and Interfaces are prefixed with the letter "I". A simple class diagram can be found in the appendix section.

1. RINGwars_7172397_Shogo_Seifert
This class serves as the entry point and controller of the main processing of the program. Here, instances of the classes are created and nested within each other.

2. IReader → StepReader
Is responsible for the reading of the step.txt files. It constructs a GameModel of the current and, if available, the previous game state.

3. GameUtils
Provides a big set of static helper functions that are required in many parts of the agent, especially the move methods. To get a better overview, the methods are organized into the subclasses **Analysis**, **Combat**, **Ring** and **MathAndRandom**. Using static methods is not fully object-oriented, but it enables rapid experimentation and offers as a lightweight library with easy access. In a revised version, the nested classes could be refactored in separated proper classes.

4. Node
A Node object represents one specific node of the ring. It stores the owner, the number of Fernies, the index as well as other fixed properties and provides some basic methods.

5. GameModel
A GameModel contains the complete (visible) current game state of a step. It defines the structure of the ring as a List<Node> object, including the owner of each node, and additionally stores the number of new Fernies, the visibility, the ring size and max Fernies.

6. StepAnalyzer
Analyzes the current and the previous game states and rates the enemy's behavior.

7. *Abstract Moves* → different **Move categories**
Move classes offer move methods that return the move in an acceptable form as a String object. They are building blocks for a combination of moves done in one step or can be used individually.

8. *AbstractStrategyDecider* → different types of **StrategyDeciders**
This is the strategic core of the agent. It decides on the current game state and analyzes of the StepAnalyzer and combines multiple moves from the move classes into a final, multi-part move.

9. OrderMap
An Utility-Enum, which controls the sorting behavior of certain move methods.

10. IMoveWriter → MoveWriter
Generates a formally correct move file. It writes the agent's final move to the move.txt file so that the game system can execute it.

11. MoveResult
MoveResult is an implementation that serves as a return type for complex methods that take Fernies from the budget iteratively. It consists of a String attribute and an int attribute.

3.3 Object Oriented Design Principles

The first question that arises is whether it is truly necessary to implement the agent in an object-oriented manner. In principle, the task could also be solved using a purely imperative approach. However, since the agent should be flexible, extensible and adaptable to different game situations, an architecture that follows some of the principles of object-oriented programming proved to be the optimal approach.

Object-oriented design helped to clearly separate different responsibilities within the program. This allowed and allows in the future, to modify and optimize individual parts of the agents or extending it without affecting the entire system.

Furthermore, the encapsulation of the fixed data in the GameModel or Node objects ensure that these data are only modified and accessed in a controlled manner. This is in particular important to guarantee a consistent game state each step and to reduce illegal moves. Inheritance and Abstraction made it possible to reduce redundant code and to define general strategy components while strategies and moves can be tested, debugged, extended and optimized independently.

Finally, modularity and reuseability were essential while developing the agent. As the strategy evolved over time, new move-types could be added, different StrategyDeciders with different decision mechanism could be tried out without the pressure to refactor existing code.

Overall, the object-oriented approach provided a robust foundation for implementing a reactive and flexible agent without having an excessive complexity of the system.

3.4 Budget System

3.4.1 Motivation

As the agent should be able to do different moves in one step, I faced the central challenge to implement this ability without losing the flexibility and robustness. Of course, I also could have written very long moves for each strategy, but this would effect the modularity and would make it harder to test and debug.

The main problem I faced was: how could the agent combine different moves without doing illegal moves and manipulating the fixed game state? In fact, each move had to know whether another move had already taken some Fernies of a node. Without the budget system, the moves don't know how many Fernies actually are left each node, which could lead to illegal moves. This is where the budget-system seemed to be a proper solution since it maintains an internal representation of the budget of each node throughout the step and "simulates" the moves before they are actually done, while ensuring consistency without mutating the global game state.

3.4.2 Implementation

The budget system is a simple HashMap ¡Integer, Integer¿, implemented as an attribute of the StrategyDecider. It stores (Node.index, Node.fernieCount) for each of my Node. The map gets updated in every step in the very beginning. At the beginning of each step, the current Budget of the owned Fernies is stored like that and is linked to the (real) index of each node on the ring. The map gets its information of the GameModel, which has the whole ring as an attribute. Each time, when the agent takes or puts Fernies on a node, this budget gets updated. The budget is not representing the actual game state, but is a "shadow state" that simulates the remaining Fernies on each node during the execution of several moves in the same step.

What the budget has to ensure is that:

- it is an internal copy of the available Fernies of each node
- each method has to update the budget of each node realistically and instantly
- all moves have to refer to the same simulated budget state
- to ensure that no Fernies are used twice

3.4.3 Challenges

Although this system was useful and necessary extension, it turned out to be complicated to handle. To correctly and consistently update the budget proved to be challenging, as numerous methods are executed sequentially in one step, and each of them can add or subtract Fernies from a node. The major challenge is the distinction between counts of Fernies that are from the original game state and values that have already been modified via the budget. It is easy to accidentally mix both sources which then leads to illegal moves.

In retrospect, the best solution would be to write a specific, centralized mechanism in a separate method or a budget handler to increase safety and to reduce the complexity of some methods further. However, at the state this issue came apparent often and clear, the code was already in an advanced state and structural change like that would have required a lot of rewriting of the program.

Despite these challenges, the system still proved to be practical and a big improvement for developing a flexible agent and avoiding illegal moves, compared to earlier versions.

3.5 Important Methods

In the appendix, the functionality of the most important methods of the agent are described in words and images or in pseudo code with the goal that the reader gets a better insight into the core elements of the agent.

3.6 Core Data structures

The agent uses four main data structures with a crucial role in different parts of the program.

3.6.1 Node

The Node class is an implemented class that serves as a data structure. An instance of this class represents a specific node on the Ring. The class should not have too many specific information that could also get easily by using a GameUtils function in cases where the information is needed for a specific purpose. This design choice follows the principle of separated responsibilities and that the data structures should not be overloaded with elements that belong to the strategy context.

3.6.2 GameModel

The GameModel class represents the current game board of a step. The GameModel bundles information about the current game board and serves only as a static data container with private attributes that is shared across different parts of the agent.

3.6.3 ArrayList

The agent uses an ArrayList all the time in the program for collections of nodes with specific characteristics. ArrayLists are very flexible and are able to adapt to the ring size of the current game. They are easy to iterate over, and access to individual elements is simple and performed in O(1) naturally via the index, while insertions or removals do not happen. Compared to other data structures like LinkedLists, Arrays or Sets, it provides a suitable balance between performance, low need of memory, flexibility and readability for the developing the agent.

3.6.4 HashMap/Budget

The HashMap is exclusively used for a crucial part of the overall functionality of my agent: the Fernie budget. Access in constant time to the values and key indexing seemed to make HashMap a good solution for the budget problem, as it uses index as the fixed part of the nodes and the number of Fernies as a simulation part and shadow state.

4 Analysis and Performance

4.1 Behavior in different Game Situations and against other agents

This section describes the behavior of the agent in different game settings against the well-known Agent 2 and several strong agents developed by fellow students. For all agents, 12 different settings plus the default settings were evaluated. Each setting was played against each opponent over 20 games. With this data, we get insights about the agents behavior and its strengths and weaknesses.

Table 2: **Different Game Settings**

Setting	Ring size	Visibility	Fernie Growth	Ownership Bonus	Max Fernies
Standard	20	5	10	5	10000
S 1.1	15	3	10	5	10000
S 1.2	15	3	10	25	10000
S 1.3	15	6	10	5	10000
S 1.4	15	6	10	25	10000
S 2.1	25	5	10	5	10000
S 2.2	25	5	10	25	10000
S 2.3	25	10	10	5	10000
S 2.4	25	10	10	25	10000
S 3.1	35	7	10	5	10000
S 3.2	35	7	10	25	10000
S 3.3	35	15	10	5	10000
S 3.4	35	15	10	25	10000

The table of the results against each agent is shown in the appendix. It would be beyond the scope to examine all factors in detail, which is why the Fernie Growth and the maximum capacity of Fernies per node stay the same and we focuses on the ring size and visibility range. First, we must note that the Agent 2 is no real challenge, which is why it will not be considered furthermore.

Analyzing the results, it seems that there is no clear dominant behavior of the agent. An increased visibility is a positive effect against Agent Y, while it has no significant change against Agent X and Agent Z. Comparing all results against all opponents, an ownership bonus of 5% has a positive impact on the agents performance than an ownership bonus of 25%, which results in fewer winning games. With a total winning rate of 79% on large rings, 60% on mid rings and, 51% on small rings, it becomes clear that the agents performs well on large rings. In summary, the ring size has the most influence on the agents performance, followed by the ownership bonus and finally the visibility.

The evaluation was performed against a limited number of agents. Consequently, the results can not be generalized. Also, minor changes of the parameters, for example changing the visibility only about 1 degree, can change the performances significantly. Therefore, these results should not be interpreted as hints and not as absolute conclusions.

4.2 Strengths and Weaknesses

The agent performs very well on larger rings and benefits partly from an increased visibility. Considering the agents aggressive behavior, attacking several enemies at the same time that have to be visible, this fits well with the overall strategy. The agent is an aggressive agent, which tends to perform well against similar, actively acting opponents. With its modularity, it would be interesting to test several other combinations or more defensive moves.

The most challenging opponents are extremely defensive enemies. Some attacks do not use enough Fernies to defeat them. As a result of the not fully developed ratings of the opponent, the agent cannot reliably detect theses situations. While dynamic change of the behavior by switching to attack more powerful or retreat to defensive passive behavior was explored, it could not be implemented in a robust way yet. Also, a calculation of the ownership bonus could improve the agents behavior, since it defines the amount of necessary expansion

From the architecture layer perspective, the agent is extremely flexible, extensible and relatively easy to debug. As a trade-off of this approach, it is sometimes not easy to trace the behavior since several moves are executing in one step.

5 Conclusion and Future Work

For me, the Programmierpraktikum was very enjoyable while I have learned a lot at the same time. I learned a lot about planning a program, about debugging, using different helpful shortcuts in Java/Eclipse, get to know a little bit of working with GitLab, handling exceptions and how basic AI systems can be implemented.

In particular, I learned how a program can develop and evolve over time, also through iterative programming , trial and error and how intensive planning with pen and paper can actually save a significant amount of time.

While the main task required to implement a single agent, my chosen design goes a little beyond by implementing in a very modular way, in which the final agent represents a concrete configuration of the components. The main advantage of this approach was robustness and the ability to experiment with different strategies without refactoring the whole program.

The resulting agent demonstrates competitive performance among a wide range of game settings while running stable, building a solid foundation for extensions. Future work, it might be interesting to experiment again with more carefully calculated and subtle decision trees to increase adaptability.

Finally, I would like to thank the support in the Moodle forum as well as my fellow students for many helpful discussions throughout the process and doing training battles.

6 Sources

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Pearson Education, Inc., 2010.

[2] A. Poetzsch-Heffter, FernUniversität in Hagen, *Einführung in die Objektorientierte Programmierung*, Skript, FernUniversität in Hagen, 2024

A Appendix

A.1 Flowchart Decision Making Agent.FINAL

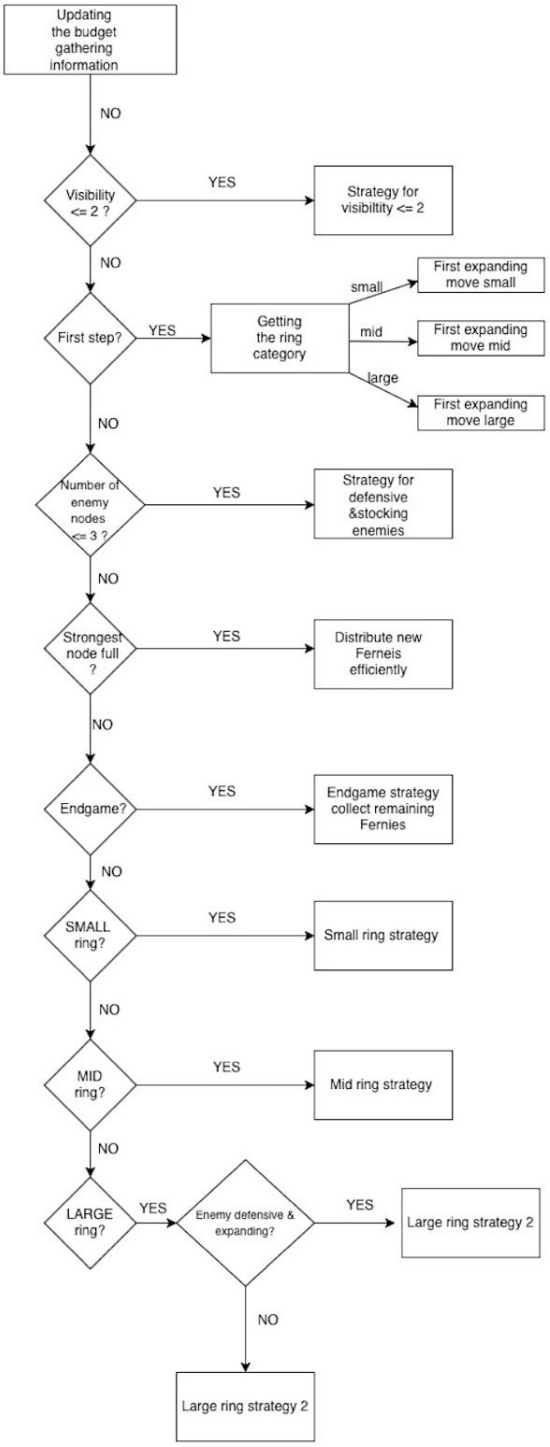


Figure 2: Flowchart of the decision making of the final agent

back

A.2 Class Overview

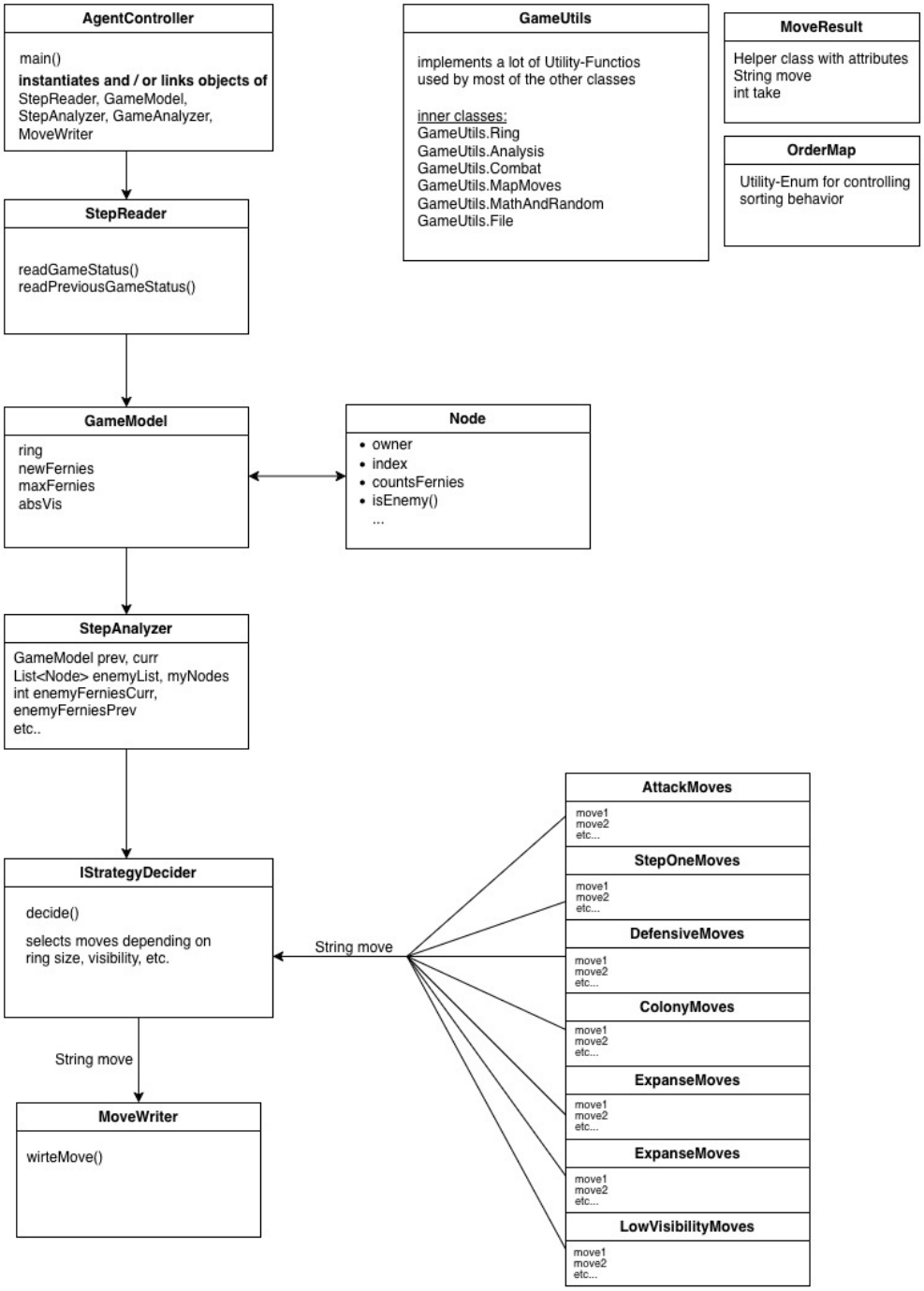


Figure 3: class overview of the agent

back

A.3 takeFerniesFromBudget()

Algorithm 1 *

takeFerniesFromBudget (budget: Map, gm: GameModel, limit: int, necessary: int, double[] take, OrderMap order)

```
Smove ← ""
rest ← necessary
sumTaken ← 0
for all t ∈ take do
    copy all Entries of the map in a ArrayList
switch (order)
sort the ArrayList dependent on the order parameter

    for all (index, budgetAvail) ∈ sorted do
        if rest ≤ 0 then
            break
        end if
        realAvail ← number of Fernies at node index
        available ← min(budgetAvail, realAvail)
        if available ≤ limit then
            continue
        end if
        candidate ← ⌈available · t⌉
        candidate ← min(candidate, available)
        took ← min(candidate, rest)
        if available − took < limit then
            continue
        end if
        append move taking took Fernies from node index
        update budget[index] ← budgetAvail − took
        rest ← rest − took
        sumTaken ← sumTaken + took
    end for
end for
```

back

A.4 getBestBorderPlacement(Node enemyNode, GameModel gm)

This method is located in the GameUtils.Combat class and is used often a move method wants to attack an enemy. It calculates the best placement to put the Fernies for attacking, while preferring border and triple fights over local fights and processing through cases for both sides.

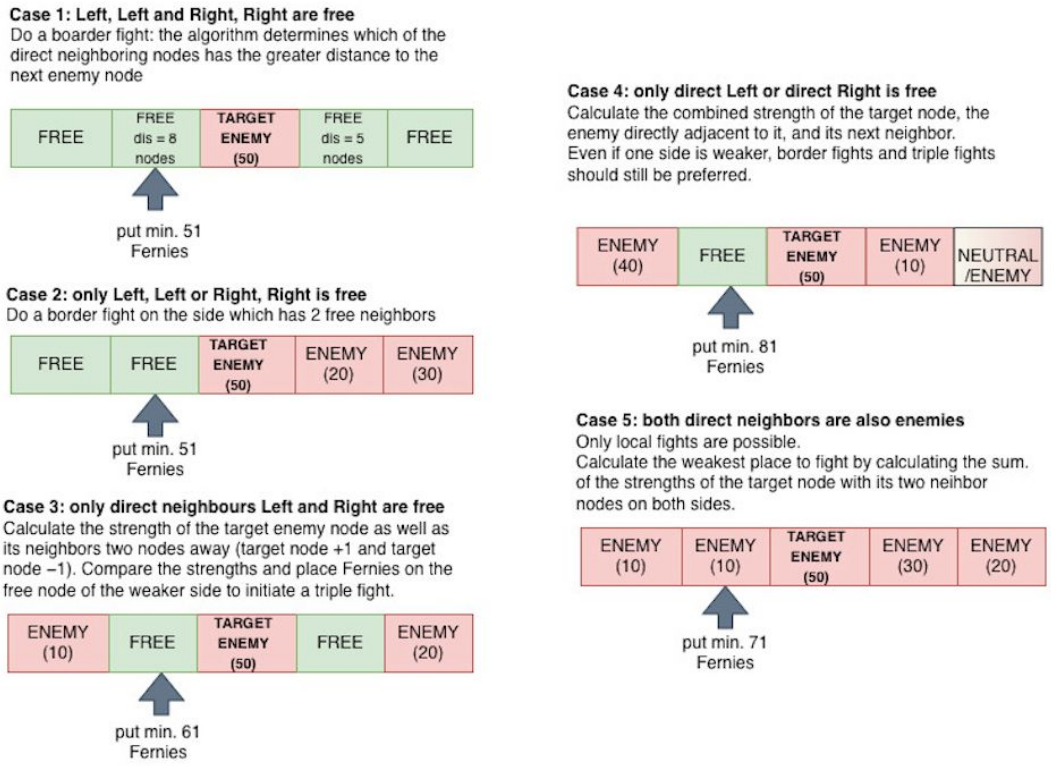


Figure 4: working of the method **getBestPlacement(Node enemyNode, GameModel gm)**

back

A.5 fightStrongestEnemiesBestPlacement()

This move fights as much enemies as possible at once from descending strength. A short abstract explanation of the methods functionality:

Algorithm 2 *

Fight All Strongest Enemies

```
sort enemy nodes by descending strength
for all enemy in enemies do
    calculate best placement
    if placement is invalid then
        continue
    end if
    calculate enemy strength and required threshold
    if available Fernies are insufficient then
        continue
    end if
    take Fernies from own budget
    place more Fernies then enemy strength
    update budget of nodes where Fernies were taken
    if no budget remains then
        break
    end if
end for
if Fernies remain unused then
    place remaining units on own node
end if
return move as String
```

back

A.6 Winning Rates

Table 3: **Winning Rates in percentage of Agent.FINAL**

SETTINGS	Agent 2	Agent X	Agent Y	Agent Z
Standard	100	90	100	0
S 1.1	90	95	100	0
S 1.2	95	100	5	25
S 1.3	80	90	10	25
S 1.4	95	60	100	0
S 2.1	100	90	60	45
S 2.2	100	80	0	45
S 2.3	100	80	95	55
S 2.4	100	60	65	40
S 3.1	100	95	95	75
S 3.2	100	90	85	45
S 3.3	100	100	100	50
S 3.4	100	55	100	60
TOTAL	**97**	**84**	**74**	**38**
OVERALL	**73**			

Table 4: **Winning Rates in percentage of Agent.KOMPLEX**

SETTINGS	Agent 2	Agent X	Agent Y	Agent Z
Standard	100	85	75	45
S 1.1	95	55	75	11, 2 Remis
S 1.2	95	90	10	8, 50, 6 Remis
S 1.3	100	55	20	8, 8 Remis
S 1.4	90	85	10	35, 6 Remis
S 2.1	100	100	5	70
S 2.2	100	90	5	35
S 2.3	100	100	100	50
S 2.4	95	45	85	10
S 3.1	100	95	100	85
S 3.2	100	85	85	60
S 3.3	100	85	100	55
S 3.4	100	65	100	65
TOTAL	**98**	**80**	**60**	**45**
OVERALL	**70**			

back to analysis back to discussion